



Instituto Superior de Engenharia

Politécnico de Coimbra

LICENCIATURA EM / DEGREE IN ENGENHARIA
INFORMÁTICA

Trabalho Prático - Programação Orientada a Objetos

Autores / Authors

Duarte Fernandes Santos 2023144379

Marcelo Rodrigues Manteigas 2023131121



INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, Dezembro 2025

(

Índices

Conteúdo

1. Introdução	5
2. Estrutura do Programa.....	6
Classe Settings.....	6
Struct Célula	6
Classe Planta	6
Classe Ferramenta.....	6
Classe Jardineiro.....	6
Classe Jardim.....	6
Classe Simulador.....	6
Classe UserInterface.....	6
3. Análise Detalhada das Classes.....	7
3.1) Simulador.....	7
3.2) Jardim e Celula.....	7
3.3) Jardineiro	7
3.4) Planta	8
3.5) Ferramenta	8
4. Decisões de Implementação em C++	9
1. Smart Pointers (std::unique_ptr).....	9
2. Biblioteca (std::vector).....	10
3. Biblioteca (std::map)	10
4. Biblioteca (std::set).....	10
5. Conclusão.....	11

1. Introdução

Este trabalho consiste num simulador de um ecossistema de jardim desenvolvido em C++. O sistema modela a interação entre um ambiente biológico (plantas, solo), um agente humano (o Jardineiro) e ferramentas. O objetivo é gerir recursos (água e nutrientes), cultivar plantas e manter o jardim através de comandos de consola.

O projeto segue o paradigma de **Programação Orientada a Objetos (POO)**, utilizando herança, polimorfismo e gestão manual e automática de memória.

2. Estrutura do Programa

Classe Settings

Classe fornecida pelo professor que define constantes e configurações estáticas, definindo regras para o Jardim, Jardineiro, Plantas e Ferramentas.

Struct Célula

Estrutura fundamental que compõe o mapa. Armazena as propriedades do solo (água e nutrientes) e pode conter ponteiros para uma Planta e uma Ferramenta.

Classe Planta

Classe base abstrata para todos os seres vivos vegetais do jardim. Define atributos como saúde (água/nutrientes), idade e comportamentos comuns (agir, clone).

Classes derivadas de Planta: classe Roseira, classe Cacto, classe ErvaDaninha e classe PlantaExotica.

Classe Ferramenta

Classe base abstrata para itens que podem ser recolhidos e usados pelo Jardineiro. Define número de série e o método de uso.

Classes derivadas de Ferramenta: classe Regador, classe TessouraPoda, classe Adubo e classe FerramentaZ.

Classe Jardineiro

Representa o agente controlado pelo utilizador. Gere a sua posição no mapa, a mochila de ferramentas (inventário) e a ferramenta atualmente equipada.

Classe Jardim

Gerir o mapa do jogo (grelha de Células) e a visualização do mesmo. Controla a posição e existência de Plantas e Ferramentas no solo.

Classe Simulador

Coordena a simulação, gere a passagem do tempo (turnos), valida regras de fim de jogo e executa os comandos processados. Controla a persistência (gravar/recuperar estados).

Classe UserInterface

Gerir a interação com o utilizador, recebendo comandos e enviando-os ao Simulador para os processar.

Em suma, Simulador é a classe controladora principal que interage com Jardim, Jardineiro e UserInterface. O Jardim é composto por uma matriz de Celula, onde residem as instâncias de Planta e Ferramenta. A classe Settings fornece parâmetros globais a todas as outras entidades.

3. Análise Detalhada das Classes

3.1) Simulador

A classe Simulador atua como o controlador principal.

- **Responsabilidades:**
 - Gere o ciclo de vida da aplicação e o loop de simulação.
 - Processa comandos fornecidos pelo utilizador (fazOpe).
 - Controla o fluxo do tempo (avanca), acionando o comportamento das plantas e o desgaste das ferramentas.
 - Gere a persistência de estados do jardim em memória (grava, recupera).
- **Gestão de Turnos:** Monitoriza ações limitadas por turno (movimentos, colheitas, plantações) através de contadores (colheContador, movimentoContador, etc.).

3.2) Jardim e Celula

O Jardim representa uma grelha bidimensional de objetos Celula.

- **Estrutura:** Utiliza uma matriz dinâmica de ponteiros (Celula**).
- **Célula:** Cada posição contém propriedades do solo (água, nutrientes) e pode conter até um ponteiro único (unique_ptr) para uma Planta e um para uma Ferramenta.
- **Funcionalidades:**
 - Validação de coordenadas.
 - Visualização textual do mapa, com prioridade de renderização (Jardineiro > Planta > Ferramenta > Solo).
 - Gestão de recursos do solo (adicionar/remover nutrientes e água).

3.3) Jardineiro

Representa o jardineiro no tabuleiro.

- **Mochila:** Possui uma mochila implementada como um `vector<unique_ptr<Ferramenta>>`.
- **Mecânica de Ferramentas:**
 - Pode apanhar ferramentas do chão ou comprar através do comando “compra”.
 - Mantém um ponteiro (ferramentaNaMao) para a ferramenta ativa.
 - As ferramentas gastam-se ou partem-se, sendo removidas da mochila quando tal acontece.

3.4) Planta

Classe base abstrata que define características base para todas as plantas.

- **Polimorfismo:** Métodos virtuais puros como `agir()`, `clone()` e `getTipo()`.
- **Atributos:** Saúde (água, nutrientes), idade, símbolo e estado de vida (viva).
- **Implementações Específicas:**
 - **Cacto:** Robusto, consome poucos recursos. Reproduz-se se os níveis de água/nutrientes forem altos. Beleza NEUTRA.
 - **Roseira:** Planta delicada e de beleza BONITA. Possui metabolismo complexo (gasta recursos internos), morre se estiver cercada ou com excesso/falta de nutrientes. Ao reproduzir-se, divide os seus recursos com a planta filha.
 - **ErvaDaninha:** Planta invasora de beleza FEIA (sendo por isso o único alvo da Tesoura de Poda). Tem um consumo de recursos muito reduzido e morre por velhice. Possui um comportamento agressivo: ao reproduzir-se, elimina qualquer planta que esteja na célula vizinha para ocupar o seu lugar.
 - **PlantaExotica:** Planta carnívora de beleza BONITA (o que a torna imune à poda). Funciona como predadora no ecossistema: a cada 5 instantes, ataca todas as plantas numa área de 3x3 à sua volta, drenando metade da água e nutrientes das vítimas para si mesma.

3.5) Ferramenta

Classe base para ferramenta.

- **Identificação:** Cada ferramenta tem um número de série único atribuído quando criado.
- **Método usar():** Retorna um booleano indicando se a ferramenta se "partiu" ou gastou após o uso.
- **Implementações Específicas:**
 - **Regador:** Tem capacidade limitada. Adiciona água ao solo. Desaparece quando vazio.
 - **Tesoura de Poda:** Remove plantas baseando-se na sua beleza. Só corta plantas com beleza FEIA (recusa-se a cortar plantas BONITA ou NEUTRA). É indestrutível (nunca retorna true no usar).
 - **Adubo:** Funciona de forma análoga ao Regador, mas para nutrientes. Possui uma quantidade limitada e aplica uma dose fixa ao solo a cada utilização. Quando o pacote fica vazio, é removido da mochila.
 - **FerramentaZ (Super Fertilizante):** É uma ferramenta especial de uso único com efeito em área. Quando utilizada adiciona 50 unidades de água e 50 de nutrientes a cada célula num raio de 2 da posição do jardineiro. Consome-se imediatamente após o primeiro uso, desaparecendo do inventário.

4. Decisões de Implementação em C++

1. Smart Pointers (std::unique_ptr)

Polimorfismo e Propriedade Exclusiva (Jardim e Celula):

- Na classe Jardim (dentro da struct Celula) e na classe Jardineiro (vetor mochila).
- O unique_ptr é usado porque o Jardim possui as plantas e ferramentas. Quando uma Planta (classe abstrata) é instanciada como Roseira, Cacto, ErvaDaninha ou PlantaExotica, precisamos de um ponteiro para tirar partido do polimorfismo (chamar o agir() correto). O unique_ptr garante que:
 1. O objeto tem um dono único (uma planta não pode estar em dois sítios ao mesmo tempo).
 2. A memória é libertada automaticamente quando o objeto "pai" (o Jardim ou o Jardineiro) é destruído.
 3. Permite a transferência de posse com std::move (ex: apanhar uma ferramenta do chão para a mochila transfere a responsabilidade de gestão de memória do Jardim para o Jardineiro).

Gestão de Estados (Simulador):

- No mapa copiasJardim e no ponteiro jardim atual.
- Permite criar e destruir instâncias completas de Jardim dinamicamente (quando se inicia um novo jogo ou se recupera um save) sem preocupações com delete manuais.

2. Biblioteca (std::vector)

Utilizado para armazenar coleções dinâmicas de elementos onde a ordem de inserção ou a iteração sequencial são importantes.

- **Mochila do Jogador:**

- `vector<unique_ptr<Ferramenta>>` mochila na classe Jardineiro.
- O número de ferramentas que o jardineiro carrega varia (pode apanhar ou partir ferramentas). O vector permite adicionar (`push_back`) e remover elementos dinamicamente, mantendo-os organizados sequencialmente para listagens.

- **Controlo de Lógica de Turno:**

- `vector<pair<int, int>>` coords no método avança do Simulador.
- Serve para criar uma "lista temporária" de todas as plantas vivas antes de as fazer agir. Isto é crucial para separar a leitura do estado do jardim da sua modificação. Se iterássemos diretamente a grelha, uma planta que nascesse na posição seguinte poderia agir no mesmo turno em que nasceu (o que seria incorreto).

3. Biblioteca (std::map)

Utilizado para associações do tipo "Chave -> Valor", permitindo pesquisas rápidas por um identificador único.

- **Sistema de Gravação:**

- `map<string, unique_ptr<Jardim>>` copiasJardim na classe Simulador.
- O utilizador dá um nome (string) à gravação (ex: "teste1"). O map associa esse nome a uma cópia completa do objeto Jardim. Garante que os nomes das gravações são únicos (chaves duplicadas substituem o valor anterior) e permite recuperar o jardim instantaneamente pelo nome.

4. Biblioteca (std::set)

Mesmas vantagens do vector, só que mantem os elementos ordenados, não há duplicados. Objeto a ser guardado, deve ser comparado com operador<, Elemento de um set, depois de inseridos são constantes.

Inicialmente planeado para gerir os IDs das ferramentas, mas essa funcionalidade foi implementada de forma mais simples usando uma variável estática que incrementa automaticamente, tornando o uso de um set desnecessário para este fim.

5. Conclusão

O desenvolvimento deste simulador permitiu aplicar de forma prática e integrada os conceitos fundamentais da Programação Orientada a Objetos (POO) em C++. O projeto cumpriu com sucesso o objetivo de modelar um ambiente dinâmico onde interagem entidades.

Do ponto de vista técnico, a implementação destacou-se pela utilização rigorosa de gestão de memória moderna. A opção exclusiva por `std::unique_ptr` revelou-se eficaz para manter um modelo de posse claro e hierárquico, onde o Jardim detém as células e estas detêm as plantas e as ferramentas, prevenindo fugas de memória sem a complexidade de gestão manual. A utilização de polimorfismo foi crucial para a escalabilidade do sistema, permitindo que comportamentos distintos como por exemplo: o “roubo” da Planta Exótica ou a invasão da Erva Daninha fossem tratados de forma uniforme pela simulação.

A arquitetura modular, separando a lógica de negócio (Simulador), a representação de dados (Jardim) e a interação (UserInterface), facilitou o desenvolvimento e a depuração do código.

O uso de contentores da STL, como `vector` e `map`, simplificou a gestão de coleções dinâmicas, nomeadamente no inventário do jardineiro e no sistema de gravação de estados em memória.

Concluindo, este trabalho prático permitiu consolidar e explorar competências técnicas no que toca a programação orientada a objetos em C++



**Instituto Superior
de Engenharia**

Politécnico de Coimbra